Embedded Data Structure
(Lecture Note 21)

Searching Algorithms (4)

2025. 5. 29.

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

KU KONKUK UNIVERSITY

- **Sequential search with a linked list**
  - **Remember the key point of the "node" structure**

    : Just contains a pointer (how can we store a key value ?)

    ☀**memcpy(t+1, key, width);**

  - **Remember the initialization of a linked list with**

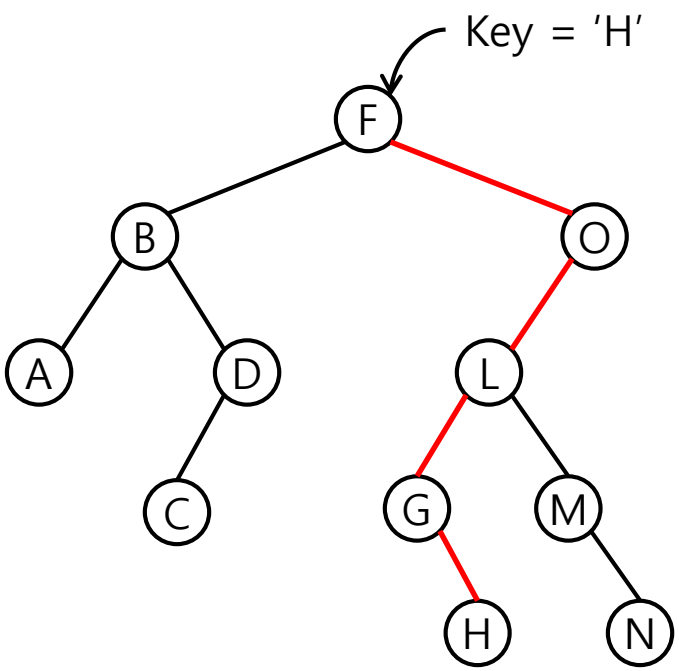
    : Draw the procedure of initialization and try to understand it !

  - **Complete your implementation with a linked list**

    - Insert and find (also delete and improved version of Seq. search)

Binary Tree Search (Make) (1/12)

- **Basic rule : left child < parent < right child**
  - **We can find the target key recursively in a tree structure**



[Example of binary tree search]

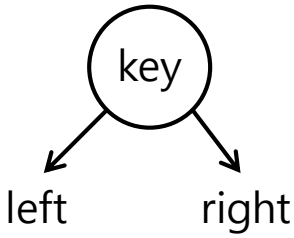
Overall procedure :

1. If key is same as the current node, then searching is complete
2. Traverse in a right direction (If key >= current node)
3. Traverse in a left direction (If key < current node)
4. If the current node is NULL, then a key is not here

Binary Tree Search (Make) (2/12)

▪ C implementation

- **Root represented by "base", which is indicated by "head"**



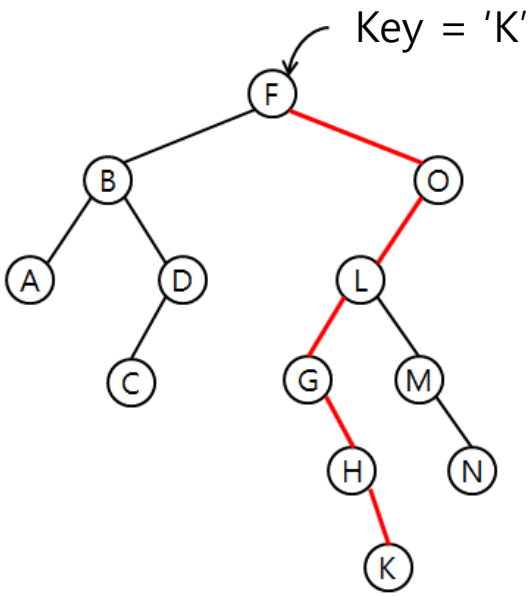

```
typedef struct _node{
    int key;
    struct _node *left;
    struct _node *right;
}node;
```

```
node *bti_search(int key, node *base, int *num)
{
    node *s;
    s = base->left;  // root node
    while(key != s->key && s != NULL)
    {
        if(key < s->key)
            s = s->left;
        else
            s = s->right;
    }
    if(s == NULL) return NULL;
    else return s;
}
```

Return a target node

Binary Tree Search (Make) (3/12)

▪ C implementation - insert



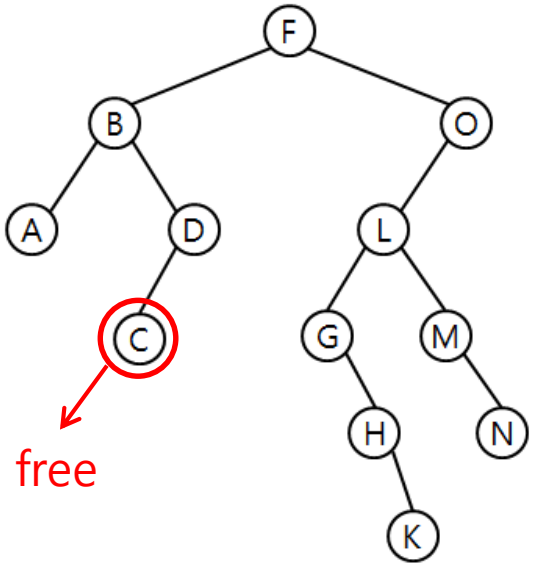
※ Insert a given key at the external node

```
node *bti_insert(int key, node *base, int *num)
{
    node *p, *s;
    p = base;
    s = base->left;
    while(s != NULL){
        p = s;
        if(key < s->key) s = s->left;
        else s = s->right;
    }
    s = (node *)malloc(sizeof(node));
    s->key = key;
    [                    ]  (two statements)
    if(key < p->key || p == base) p->left = s;
    else p->right = s;
    (*num)++;
    return s;
}
```

Binary Tree Search (Make) (4/12)

- **Deletion of binary tree**

  - **Case 1: external node**

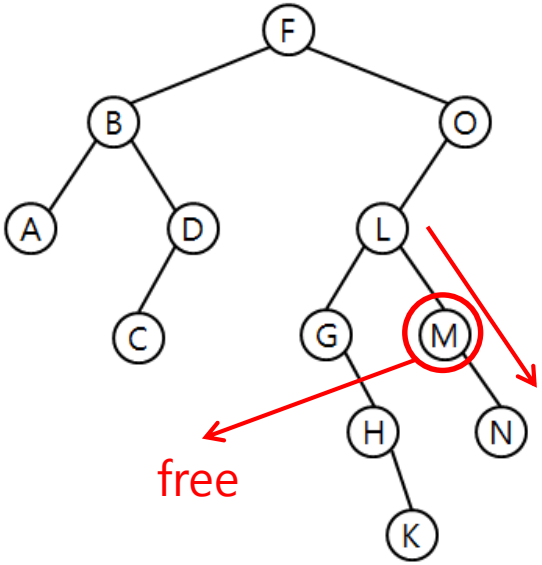


For example, delete node "C"

→ Left child of D is set to NULL

→ Node C is free

  - **Case 2: one child**



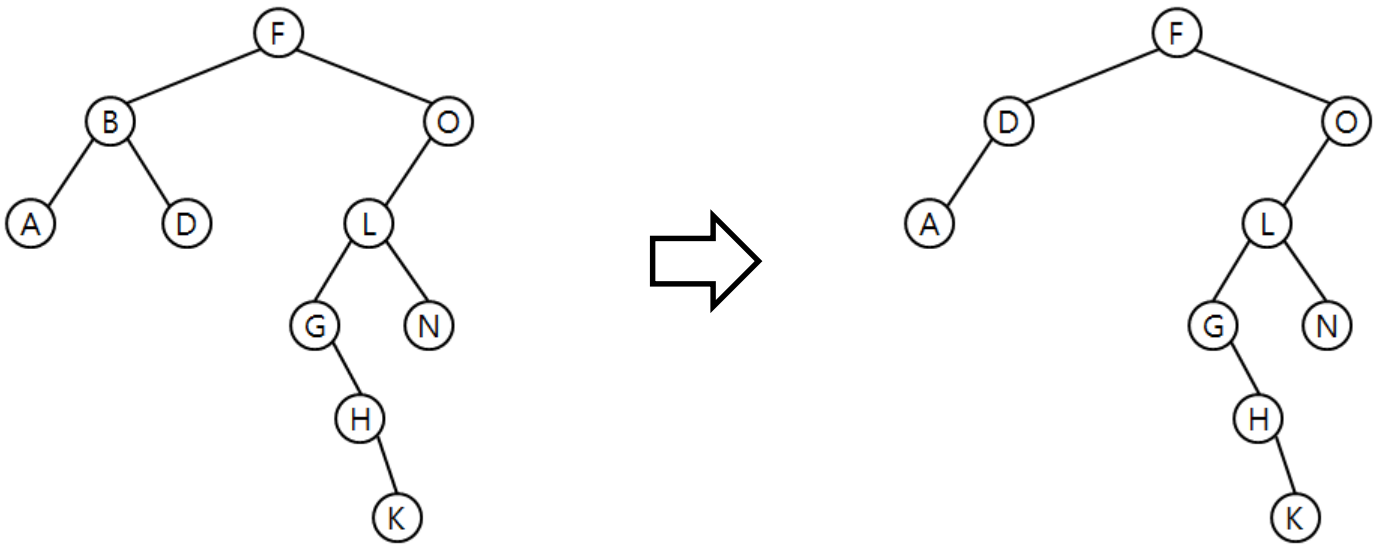
For example, delete node "M"

→ L->right indicates N

→ Node M is free

Binary Tree Search (Make) (5/12)

- **Deletion of binary tree – cont'd**
  - **Case 3-1 : child node is full (not root)**

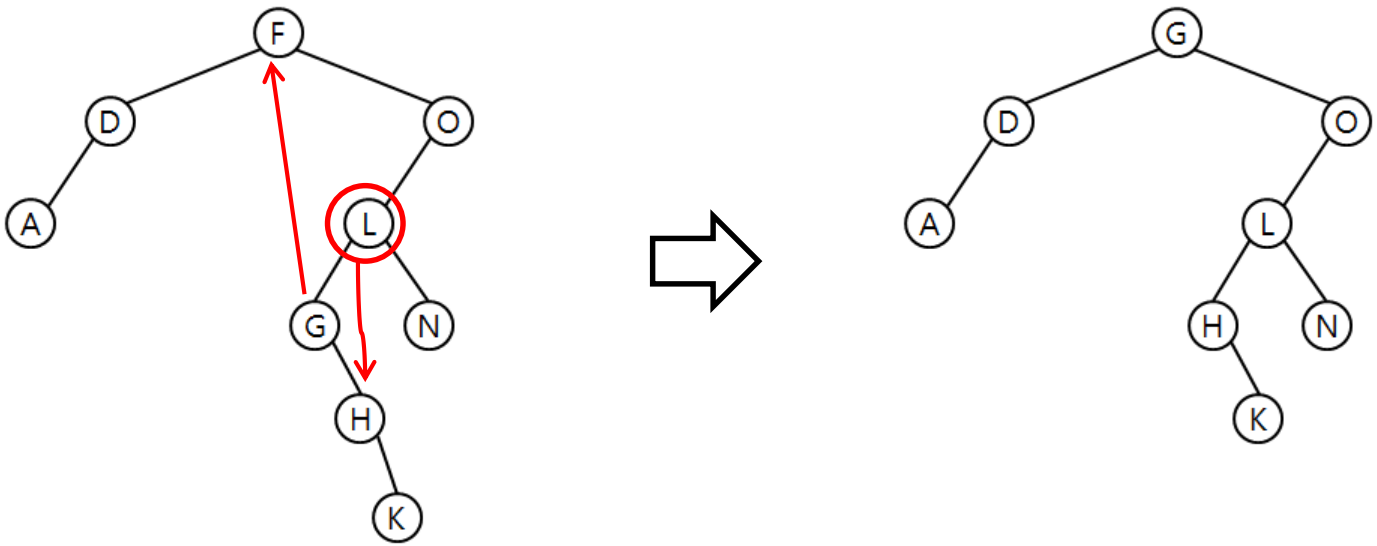


For example, delete node "B"

→ Node D becomes parent (select larger one : D > A)

→ F->left indicates the node D

- **Deletion of binary tree – cont'd**
  - **Case 3-2 : root**



For example, delete node "F" (root)

→ Select "D" or "G" instead of "O" **(Why ?)**

→ Select "G" (G > D) and find "G" using the left nodes of "O"

→ Link btw parent of F (i.e., head) and "G"

Binary Tree Search (Make) (7/12)

KU KONKUK UNIVERSITY

▪ C implementation of deletion

- **Based on three cases and pointers**

```
node *bti_delete1(int key, node *base, int *num)
{
    node *parent, *son, *del, *nexth;
    parent = base;
    del = base->left;
    while(key != del->key && del != NULL){
        parent = del;
        if(del->key > key) del = del->left;
        else del = del->right;
    }
    if(del == NULL) return NULL;

    if(del->left == NULL && del->right == NULL)
        son = NULL;
    ...
```

→ **Case 1**

del : delete node

parent : parent of del

son : child of parent

nexth : parent of son (for case 3)

Binary Tree Search (Make) (8/12)

KU KONKUK UNIVERSITY

- **C implementation of deletion – cont'd**
  - **Based on three cases with pointers (node structure)**

```
else if(del->left != NULL && del->right != NULL)
{
    nexth = del->right;
    if(nexth->left != NULL){
        while(nexth->left->left != NULL) next = next->left;
        son = nexth->left;
        nexth->left = son->right;
        son->left = del->left;
        son->right = del->right;
    }
    else
    {
        son = nexth;
        son->left = [          ];
    }
}
```

Case 3

Case 3-2 (root)

Case 3-1

KU KONKUK UNIVERSITY

▪ C implementation of deletion – cont'd

- **Based on three cases with pointers (node structure)**

```
    else
    {
        if(del->left != NULL) son = del->left;
        else son = del->right;
    }
    if(key < parent->key || parent == base)
        parent->left = son;
    else
        parent->right = son;

    free(del);
    (*num)--;

    return parent;
}
```

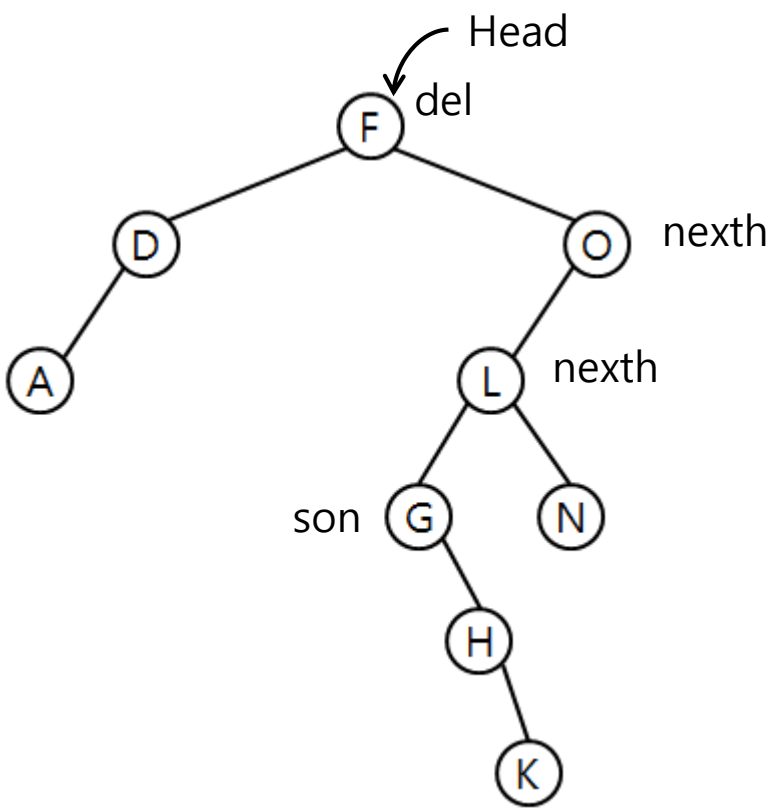
Case 2

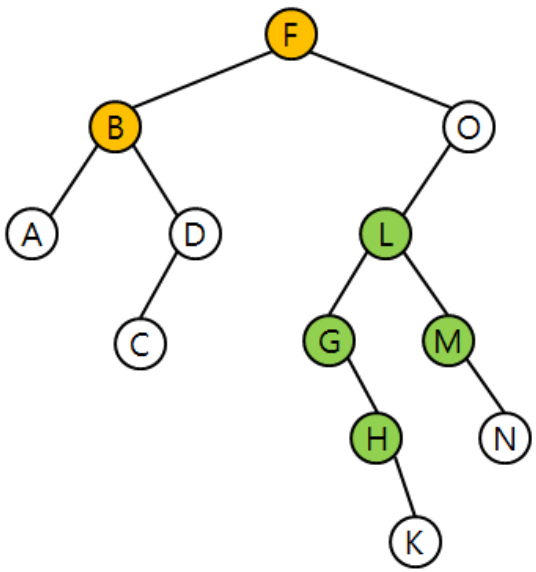
What does it mean ?

Return "parent" of deleted node

Binary Tree Search (Make) (10/12)

- **Pictorial example and improved version**
  - **Make compact the given cases as follows:**





1. No right child
2. No right->left child
3. Otherwise (i.e., case 3)

Binary Tree Search (Make) (11/12)

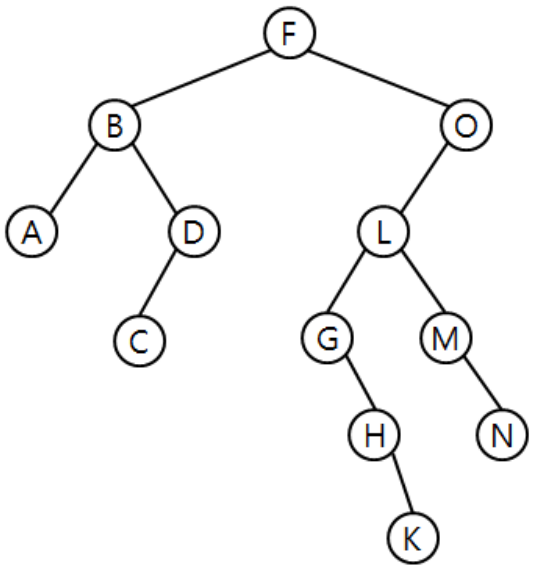
- **C implementation of improved version**
  - **Reduce the number of conditions (i.e., if)**

```
node *bti_delete(int key, node *base, int *num)
{
    /* same with bti_delete1()  for finding del node */
    if(del->right == NULL) son = del->left;
    else if(del->right->left == NULL){
        son = del->right;
        son->left = del->left;
    }
    else{
        nexth = del->right;
        while(nexth->left->left != NULL) nexth = nexth->left;
        son = nexth->left;
        netxh->left = son->right;
        son->left = del->left;
        son->right = del->right;
    }
    /* same with bti_delete1()  for making connections with new parent */
}
```

Binary Tree Search (Make) (12/12)

- **Implementation example**
  - **Make a following tree and print out the given tree**

    ※ Hint : using the insert() function



[Binary tree]

Make your code complete !

Summary

- **Make a binary tree for searching**
  - **Remember the rule to traverse (find) a given tree**
  - **Understand insert operation to make a tree and deletion**